

Generative AI (GenAI)

Fundamentals — Complete Notes

Table of Contents

1. [What is Generative AI](#)
 2. [Generative vs Discriminative Models](#)
 3. [Neural Network Foundations \(Quick Refresher\)](#)
 4. [The Transformer Architecture](#)
 5. [Large Language Models \(LLMs\)](#)
 6. [How LLMs Are Trained](#)
 7. [Prompt Engineering](#)
 8. [Retrieval-Augmented Generation \(RAG\)](#)
 9. [AI Agents](#)
 10. [Other Generative Model Families](#)
 11. [Limitations, Risks & Ethics](#)
 12. [Real-World Applications](#)
 13. [Quick-Revision Cheat Sheet](#)
 14. [Common Interview Questions](#)
-

1. What is Generative AI

Generative AI refers to AI systems that **create new content** — text, images, audio, code, or video — rather than simply classifying or predicting a label for existing data. It learns the underlying patterns and structure of training data well enough to produce **original, plausible outputs** that resemble that data.

- Learns the probability distribution of data (e.g., "what does natural language typically look like?").
- Generates *new* instances by sampling from that learned distribution.
- Powers tools like ChatGPT/Claude (text), Midjourney/DALL-E (images), GitHub Copilot (code).

Why It Matters Now

- Modern generative systems (LLMs, diffusion models) reached a quality/scale threshold where outputs are useful for real enterprise work — not just research demos.
 - Industries actively applying it: enterprise software (copilots, chatbots), healthcare (drug discovery, report summarization), finance (fraud pattern synthesis, report generation), aviation, and more.
-

2. Generative vs Discriminative Models

This is one of the most frequently asked **conceptual** questions in GenAI interviews.

Aspect	Discriminative Models	Generative Models
Goal	Learn the boundary <i>between</i> classes	Learn the <i>distribution</i> of the data itself
Question answered	"Given this input, what label does it belong to?"	"What would a plausible new example look like?"
Output	A label, category, or score	New data — text, image, audio, etc.
Examples	Logistic Regression, SVM, standard CNN classifiers	GPT, Claude, GANs, Diffusion Models, VAEs

Typical Task

Spam detection, image classification, fraud detection

Text generation, image synthesis, code generation

Simple way to explain it in an interview: "A discriminative model draws a line between cats and dogs. A generative model learns what a cat *looks like* well enough to draw a brand-new one that never existed."

3. Neural Network Foundations (Quick Refresher)

You don't need deep math for a fresher interview, but you should be comfortable with these terms since they underpin every LLM explanation:

- **Neural Network:** Layers of interconnected nodes ("neurons") that transform input data through weighted connections to produce an output.
 - **Parameters (Weights):** The learned numeric values that determine how strongly one neuron's output influences the next layer. LLMs with "70 billion parameters" means 70 billion such learned numbers.
 - **Activation Functions:** Introduce non-linearity so the network can learn complex patterns (common ones: ReLU, Sigmoid, GELU).
 - **Loss Function:** Measures how wrong the model's prediction was compared to the correct answer.
 - **Backpropagation + Gradient Descent:** The process of adjusting weights backward through the network to reduce the loss, repeated over many iterations ("training").
 - **Overfitting vs Underfitting:** Overfitting = model memorizes training data but fails on new data. Underfitting = model is too simple to capture the pattern at all.
-
-

4. The Transformer Architecture

Transformers are the neural network architecture that replaced older RNN/LSTM-based models for most language tasks, and they are the foundation of every modern LLM (GPT, Claude, Gemini, Llama, etc.).

4.1 Why Transformers Replaced RNNs/LSTMs

- RNNs/LSTMs process text **sequentially** (word by word) — slow, and they struggle to "remember" relationships between words that are far apart in a sentence.
- Transformers process an entire sequence **in parallel** and use a mechanism called **self-attention** to directly relate any word to any other word, regardless of distance.

4.2 Self-Attention

Self-attention lets each token "look at" every other token in the input and decide how much to focus on each one when building its own representation. This is what allows a model to resolve something like "it" in *"The trophy didn't fit in the suitcase because **it** was too big"* — attention lets the model weigh which earlier word "it" most likely refers to.

4.3 Key Architectural Components

Component	Role
Tokenization	Splits raw text into smaller units (tokens) — often subwords. E.g., "programming" → "program" + "ming".
Embeddings	Converts each token into a dense numeric vector that captures semantic meaning (similar words → similar vectors).

Positional Encoding	Since transformers process tokens in parallel (no inherent sense of order), positional encoding injects information about <i>where</i> each token sits in the sequence.
Self-Attention Layers	Compute relationships/relevance between all tokens in the input.
Encoder	Reads and understands the <i>entire</i> input at once (used in models like BERT — good for understanding/classification tasks).
Decoder	Generates output one token at a time , using previously generated tokens as context (used in models like GPT — good for generation tasks).
Encoder-Decoder	Uses both — reads input fully, then generates output conditioned on it (used in translation-style models like T5).

4.4 Encoder vs Decoder vs Encoder-Decoder — Quick Map

Type	Example Models	Best For
Encoder-only	BERT	Classification, sentiment analysis, embeddings
Decoder-only	GPT, Claude, Llama	Open-ended text generation, chat
Encoder-Decoder	T5, BART	Translation, summarization

5. Large Language Models (LLMs)

A **Large Language Model** is, at its core, a **next-token prediction**

machine: given a sequence of tokens, it predicts the most probable next token — then repeats this process to generate a full response, one token at a time.

5.1 Core Concepts

- **Context Window:** The maximum number of tokens (input + output combined) the model can "see" at once. A longer context window means the model can consider more surrounding text before responding.
- **Parameters:** The learned weights of the network — generally, more parameters can mean more capacity to capture complex patterns (though not always better in practice).
- **Embeddings (revisited):** Not just for input — the internal vector representations that let the model reason about meaning and similarity.
- **Hallucination:** When an LLM generates plausible-sounding but factually incorrect or fabricated information. This happens because the model is fundamentally predicting *likely* text, not verifying *true* text.

5.2 Well-Known LLM Families (for context, not required memorization)

GPT (OpenAI), Claude (Anthropic), Gemini (Google), Llama (Meta), Mistral — all decoder-based transformer models, differing in scale, training data, and alignment techniques.

6. How LLMs Are Trained

LLM training typically happens in stages — a very common interview question is "walk me through how an LLM like ChatGPT is actually built."

6.1 Pretraining

- The model is trained on a **massive** corpus of internet-scale text (books, websites, code, etc.).
- Task: simple next-token prediction, repeated billions of times.
- Result: a "base model" with broad language understanding, but not yet good at following instructions or being helpful/safe.

6.2 Fine-Tuning

- The base model is further trained on a **smaller, curated dataset** for a specific purpose or domain.
- Can be **instruction tuning**: training the model on (instruction → ideal response) pairs so it learns to actually *follow* what users ask, rather than just continuing text.

6.3 RLHF (Reinforcement Learning from Human Feedback)

- Human reviewers rank multiple model outputs from best to worst for the same prompt.
- These rankings train a **reward model**, which is then used to further fine-tune the LLM via reinforcement learning — nudging it toward outputs humans actually prefer (helpful, honest, safe).
- This is the stage that turns a raw "text predictor" into an assistant-like chatbot.

6.4 Chat Templates

- Structured formatting (e.g., marking "system," "user," and "assistant" turns) that tells the model how to interpret a multi-turn conversation consistently.

Summary flow:

Massive raw text → Pretraining (next-token prediction)

- Fine-tuning / Instruction tuning (learn to follow instructions)
- RLHF (align with human preference for helpfulness)
- Deployed chat-ready model

7. Prompt Engineering

Prompt Engineering is the practice of designing inputs (prompts) to guide an LLM toward producing the most accurate, relevant, and useful output — without changing the model's underlying weights.

7.1 Why It Matters

Since LLMs are next-token predictors, *how* you phrase a request measurably changes the quality of the output. Good prompting effectively "sets up" the model's internal probability distribution to favor the response you actually want.

7.2 Common Techniques

Technique	What It Does
Zero-shot prompting	Ask the model to do a task with no examples — relies purely on its pretrained knowledge
Few-shot prompting	Provide a few examples of the task within the prompt before asking the real question — helps the model infer the pattern/format expected
Chain-of-Thought (CoT) prompting	Ask the model to reason step-by-step before giving a final answer — improves performance on multi-step logic/math problems
Role prompting	Assign the model a persona/role ("You are an expert Python developer...") to shape tone and focus

Structured output prompting	Explicitly ask for a specific format (JSON, table, bullet list) to make output easier to parse programmatically
------------------------------------	---

8. Retrieval-Augmented Generation (RAG)

RAG is a technique that improves LLM output accuracy by giving the model access to **external, up-to-date, or private information** at the time of answering — instead of relying solely on what it memorized during training.

8.1 The Core Problem RAG Solves

- LLMs only "know" what was in their training data, up to a fixed cutoff date.
- They can't natively access private company documents, live data, or anything published after training.
- Retraining a model just to add new knowledge is extremely expensive and slow.

RAG's fix: keep the LLM's weights unchanged, but *retrieve* relevant external information at query time and feed it into the prompt as extra context.

8.2 How RAG Works — Step by Step

1. **User submits a query/prompt.**
2. **Retrieval:** A retrieval system (functioning like a search engine) searches a knowledge base — often using **semantic/vector search** — to find the most relevant chunks of information.
3. **Augmentation:** The retrieved content is merged with the original user prompt, creating an "augmented prompt" that carries both the user's intent *and* the extra grounding context.
4. **Generation:** The LLM generates its final response using this

enriched prompt — producing an answer grounded in real, retrieved information rather than pure memory.

8.3 Key Components

Component	Role
Knowledge Base	The external data repository (documents, database, wiki, etc.)
Embeddings + Vector Database	Documents are converted into vector embeddings and stored so they can be searched by <i>semantic similarity</i> , not just keyword match
Retriever	The component that finds and returns the most relevant chunks for a given query
Generator (the LLM)	Produces the final answer using the retrieved context

8.4 Why RAG Matters

- Reduces **hallucination** — answers are grounded in real retrieved data.
- Keeps information **current** without retraining the model.
- Lets a company use an LLM safely over its **own private documents** (internal wikis, policies, codebases) without exposing that data during training.

Interview one-liner: "RAG is like giving an open-book exam to an LLM — instead of relying purely on memory, it looks up the relevant page first, then answers."

9. AI Agents

An **AI Agent** extends an LLM from simply *answering questions* to actually **executing multi-step tasks** — often by giving it access to external tools, APIs, or actions it can take.

9.1 What Makes Something an "Agent" vs a Chatbot

A plain chatbot responds with text. An agent can:

- **Plan** a sequence of steps needed to complete a goal.
- **Use tools** (web search, code execution, calling an API, querying a database) to gather information or take action.
- **Observe** the result of each action and decide the next step.
- **Loop** through this plan → act → observe cycle until the task is complete.

9.2 Typical Agent Loop

```
User goal → Agent plans steps → Agent calls a tool  
→ Agent observes tool's result → Agent decides n  
→ ... repeats ... → Final answer/action delivered
```

9.3 Example

Instead of just answering "what's the weather in Chennai," an agent handling "book me the cheapest flight to Chennai next Friday" would need to: search flights (tool call) → compare prices (reasoning) → check the user's calendar for conflicts (another tool call) → confirm before booking (action). This chaining of reasoning + tool use is what defines agentic behavior.

9.4 RAG vs Agents (a common point of confusion)

- **RAG** is about *retrieving information* to improve an answer.
- **Agents** are about *taking multi-step actions*, of which retrieval can be just one tool among many.

- In fact, when retrieval itself is treated as a callable "tool" within a broader reasoning loop, a RAG system essentially becomes a simple agent.

10. Other Generative Model Families

While LLMs (transformer-based, text-focused) get the most attention, GenAI covers other architectures too — good to know at a high level:

Model Type	What It Generates	How It Works (high level)	Examples
Diffusion Models	Images (and increasingly video/audio)	Learns to reverse a gradual "noise-adding" process — starts from random noise and iteratively refines it into a coherent image	Midjourney, DALL-E, Stable Diffusion
GANs (Generative Adversarial Networks)	Images, synthetic data	Two networks compete: a Generator creates fake samples, a Discriminator tries to detect fakes — both improve through this competition	Early deepfake/ image-synthesis tools
VAEs (Variational Autoencoders)	Images, structured data	Compresses data into a latent space and learns to reconstruct/ generate variations from it	Data compression, anomaly detection

11. Limitations, Risks & Ethics

Interviewers often ask a "what are the challenges of GenAI" question to test balanced understanding, not just enthusiasm.

- **Hallucination:** Generating confident but false information — a direct consequence of the model predicting *plausible* text rather than verified facts.
 - **Bias:** Models can reflect and amplify biases present in their training data.
 - **Knowledge Cutoff:** A model only "knows" information up to when its training data was collected (RAG helps mitigate this).
 - **Data Privacy:** Careful handling required when connecting LLMs to private/sensitive company data (especially relevant in RAG/agent setups).
 - **Prompt Injection:** A security risk where malicious instructions hidden in retrieved content or user input attempt to override the model's intended behavior — an active concern in RAG and agentic systems.
 - **Cost & Compute:** Training and running large models requires significant computational resources.
-

12. Real-World Applications

- **Enterprise Software:** AI copilots/assistants embedded into internal tools (e.g., code assistants like GitHub Copilot).
- **Customer Support:** RAG-powered chatbots that answer using a company's actual documentation.
- **Healthcare:** Summarizing patient reports, assisting in drug-discovery research (e.g., generating candidate molecular structures).
- **Finance:** Automated report generation, fraud-pattern analysis.
- **Software Development:** Code generation, code review

assistance, automated documentation.

13. Quick-Revision Cheat Sheet

Concept	One-Line Definition
Generative AI	AI that creates new content by learning the underlying data distribution
Discriminative Model	Learns to classify/label existing data (the boundary between classes)
Transformer	Neural architecture using self-attention to relate all tokens in a sequence in parallel
Self-Attention	Mechanism letting each token weigh its relevance to every other token
Tokenization	Splitting text into subword units the model can process
Embedding	A dense numeric vector representing a token's/ document's meaning
Context Window	Max number of tokens an LLM can consider at once
Pretraining	Initial training on massive raw text via next-token prediction
Fine-tuning	Further training on a smaller, task/domain-specific dataset
RLHF	Using human preference rankings to align model outputs with what humans actually want
Hallucination	Confident but factually incorrect model output
Prompt Engineering	Designing inputs to guide better LLM outputs without changing model weights

Few-shot Prompting	Giving the model a few examples in the prompt before the real task
Chain-of-Thought	Prompting the model to reason step-by-step before answering
RAG	Retrieving external/private data at query time to ground and improve LLM answers
Vector Database	Stores embeddings so documents can be searched by semantic similarity
AI Agent	An LLM-driven system that plans, uses tools, and takes multi-step actions toward a goal
Diffusion Model	Generates images by learning to reverse a noise-adding process
GAN	Generator vs Discriminator networks competing to produce/detect realistic fakes
